# MARÍA SÁNCHEZ DOMÍNGUEZ

COMPUTATIONAL DESIGNER AND TECHNICAL EDUCATOR | BIM, AUTOMATION, AEC WORKFLOWS

CURRICULUM VITAE

# María Sánchez Domínguez

Computational designer and technical educator | BIM, automation and AEC workflows

Barcelona, Spain · maria.sanchez.arch@gmail.com · mariasanchez.dev · LinkedIn

## PROFILE

Architect and computational designer with 5+ years in AEC, combining BIM and automation work with teaching Revit, Dynamo, Grasshopper and Rhino.Inside to professionals and university students. Experienced in understanding client workflows, scoping technical solutions and training the people who use them. Front-end (Vue.js, Vite) and Python experience; currently building foundations in cloud APIs and Autodesk Platform Services.

## EDUCATION

### MaCAD, Advanced Computation and Design - IAAC

2025–2026 · Awarded with honours

### Master's in Architecture - ETSAM, UPM

2020–2021

### Bachelor's in Architecture - ETSAB, UPC

2014–2020

## LANGUAGES

Spanish and Catalan (native) · English (C2, Cambridge CPE) · German (C1) · French (B2) · Korean (B2) · Mandarin (A2)

## EXPERIENCE

### Computational Designer / Technical Lead - DataB GmbH

05/2026 – 10/2026 · Automation and AI for manufacturing workflows (Rhino/Grasshopper, Flask, Vue.js); client advisory and delivery

### Computational Designer / Project Manager - Modelical

02/2023 – 08/2025 · BIM and parametrisation projects incl. stadium infrastructure; fabrication modelling; Revit/Rhino.Inside coordination

### Architect - MRA, Manuel Ruisánchez Arquitectos

10/2021 – 02/2023 · Basic and executive projects, tenders; Revit and Rhino

### Architect - Sagrada Família Basilica

01/2019 – 07/2020 · Heritage construction site; Rhino and Grasshopper

### Architect - DFT Arquitectes

07/2018 – 01/2019

### Competition collaborator - Gina Barcelona Architects

06/2018 – 07/2018 · Neubrandenburg Intergenerational Center, 1st prize

## TEACHING

### Instructor - Zigurat Institute of Technology

10/2025 – present · Grasshopper, Dynamo, Revit, Rhino.Inside, Python

### Seminar teacher - ETSAB, UPC

10/2025 – 06/2026 · Architectural Representation IV

### Seminar teacher - MPDA, ETSAV, UPC

10/2024 – present · Building Information: Interoperability

### Teaching assistant - MaCAD, IAAC

01/2024 – present · Integrative Modelling Seminar

## SOFTWARE

Rhino, Grasshopper, Revit + Rhino.Inside, AutoCAD, Navisworks, ACC/Forma, Lumion, Enscape, Adobe Suite, SketchUp

Python, Vue.js, Vite, HTML/CSS, Git, Postman

## CERTIFICATIONS

Autodesk Forma (Autodesk Learning Center), 2026 · Postman API Beginner and Auth, 2026 · DataCamp Intermediate Python, 2026 · IBM Python 101 for Data Science, 2025

***CONTENTS***


| | | | |
|---|---|---|---|
| 01 | Model-driven fabrication | Professional \| Modelical | 04 |
| 02 | BIM native terrain modelling | Professional \| Modelical | 06 |
| 03 | Computational bucketing | Academic \| MaCAD \| IAAC | 08 |
| 04 | Model data geometrization | Academic \| MaCAD \| IAAC \| Own APS work | 10 |
| 05 | Personal comfort copilot | Academic \| MaCAD \| IAAC | 13 |

PROJECT 01 | PROFESSIONAL | MODELICAL

# Model-driven fabrication

One Revit model that writes its own fabrication files

### PROBLEM

The client needed a precise Revit model of a stadium's façade structure, and then fabrication models for every piece, normally built in separate software.

### APPROACH

I automated both steps with Rhino.Inside Revit and Grasshopper: a full BIM model for clash detection, and scripts that read Revit parameters to export fabrication STEP files.

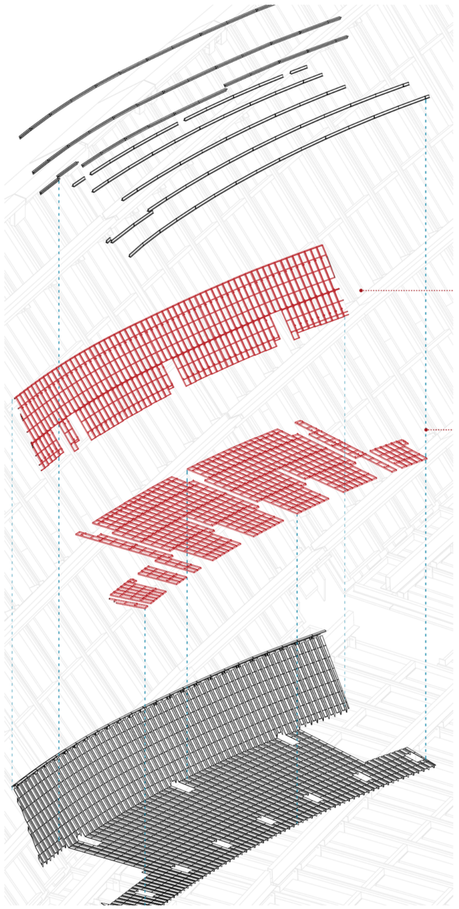
### RESULT

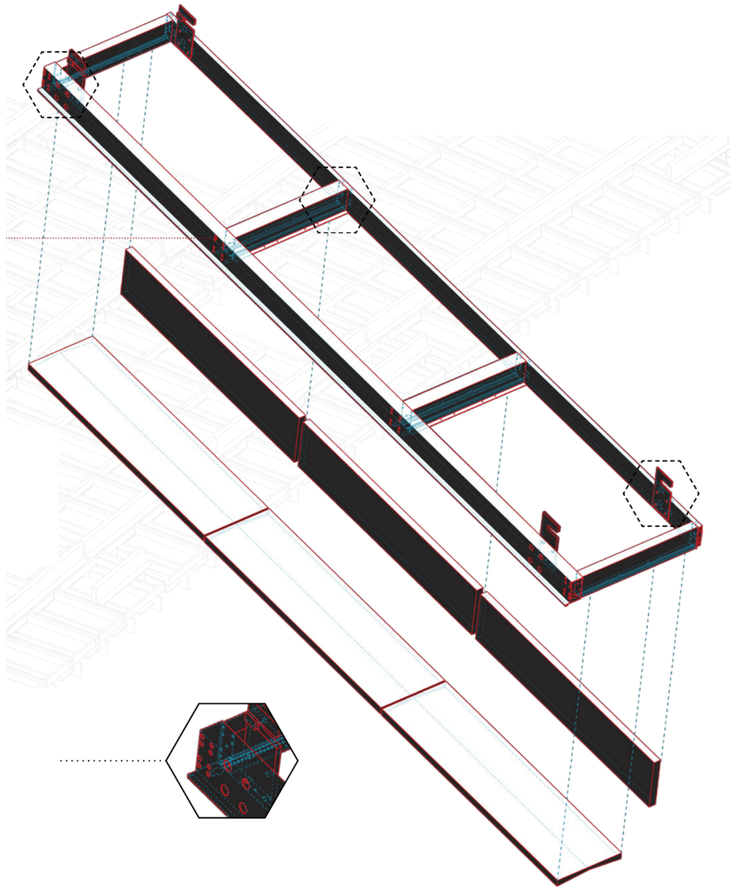
When the Revit model changed, the fabrication files updated with it, and no separate fabrication software was needed.

### COORDINATION

I led the modelling and parametrisation of the fabrication parts, their transfer to the live Revit models, and clash detection. I worked directly with the client, under supervision, and a Grasshopper definition generated the STEP files for fabrication.

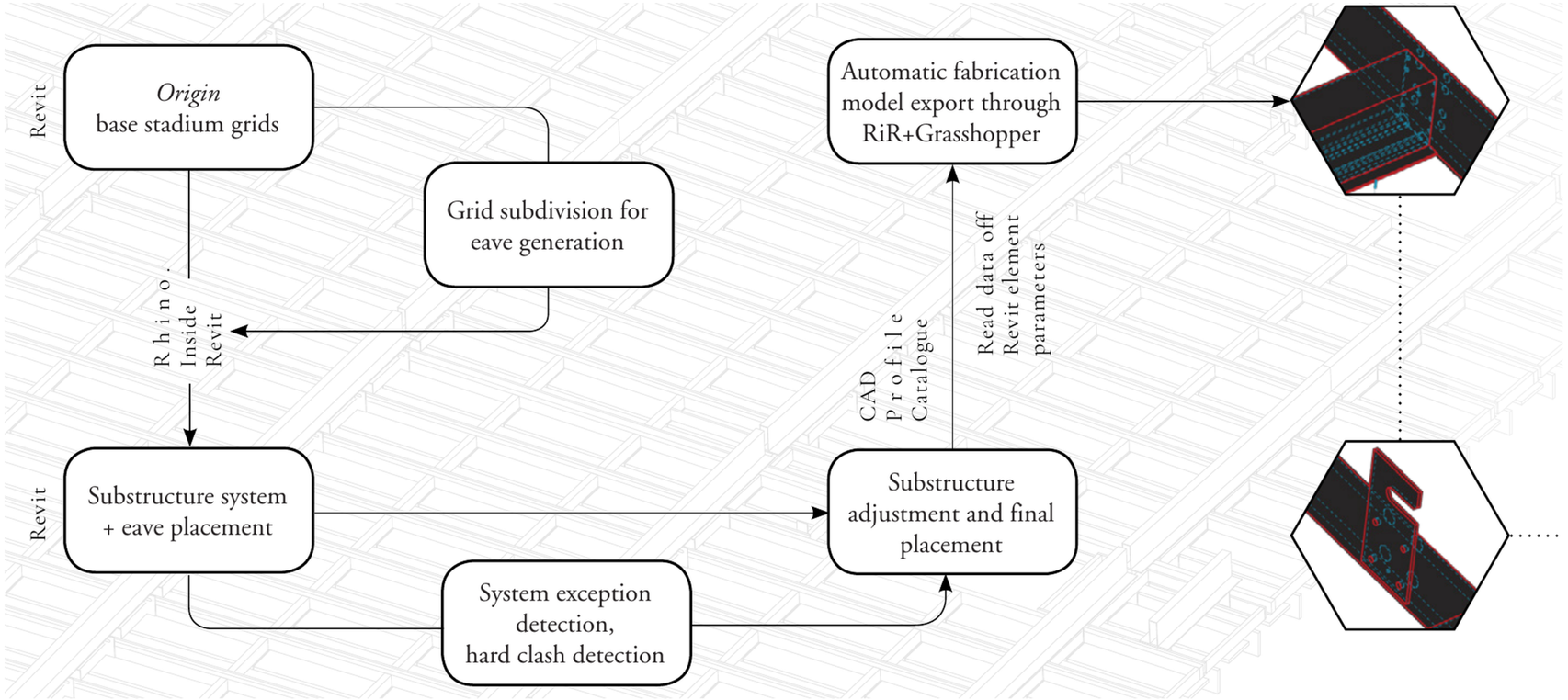
Revit, Rhino.Inside Revit, Grasshopper · 2023–2025 · Details under NDA





# From stadium grids to fabrication-ready substructure

Revit → Rhino.Inside + Grasshopper → STEP




### THE BRIEF

It began as advice on a precise Revit model of the façade structure, then grew into two tasks: a complete BIM model for clash detection, within the system and against other disciplines like the main structure, and fabrication models reading their data from it.

### THE WORKFLOW

From the base stadium grids, Grasshopper subdivided the eaves and placed the substructure through Rhino.Inside Revit. Exceptions and hard clashes were detected, then every piece was adjusted and placed for good.

### FABRICATION

Export scripts read each Revit element's parameters and a profile catalogue to write the STEP files. When fabrication raised an issue, the model changed and the files were regenerated, with no separate software like SolidWorks.

PROJECT 02 | PROFESSIONAL | MODELICAL

# BIM native terrain modelling

Turning a terrain mesh into native, editable Revit topography, floors and drainage

**PROBLEM**

Planners and architects on a live project needed the island's earthworks and urbanism in Revit, beyond what native terrain tools handle well.

**APPROACH**

Terrain built in Civil 3D and Rhino, then brought into Revit as native topography, floors, drainage and railings through Rhino.Inside and Grasshopper.

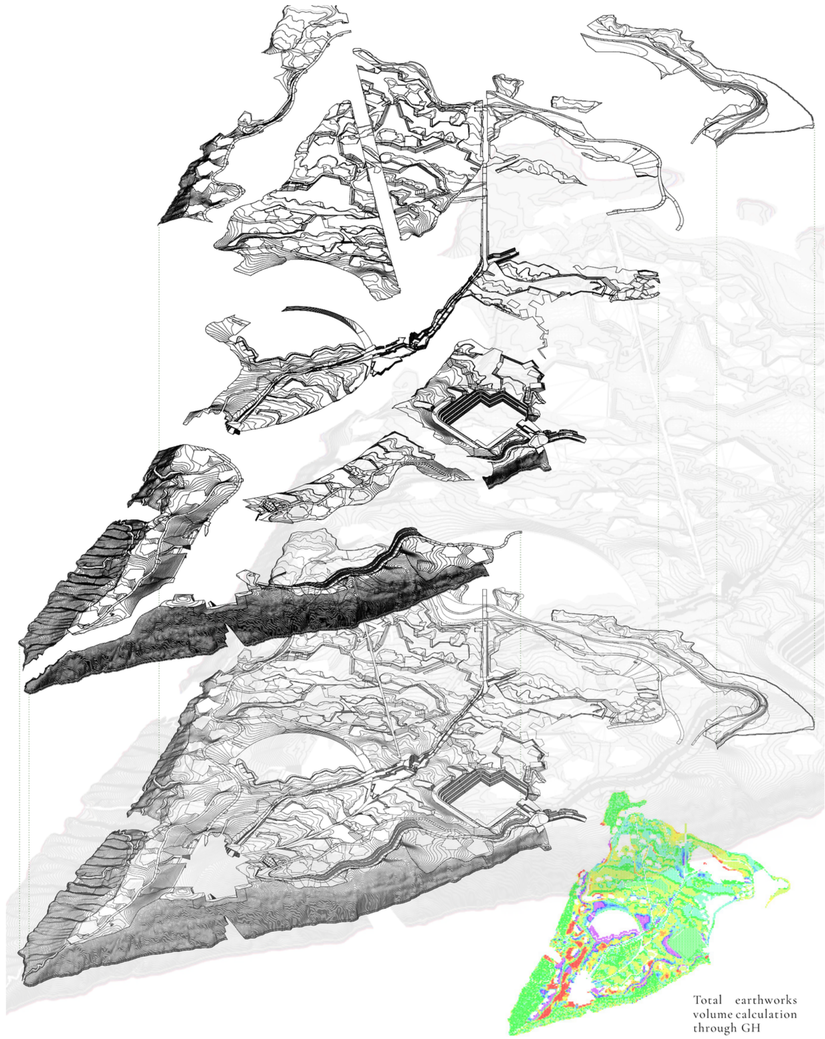
**RESULT**

One live Revit model with terrain, roads, paving and urban systems, and total earthworks volume calculated in Grasshopper.

**COORDINATION**

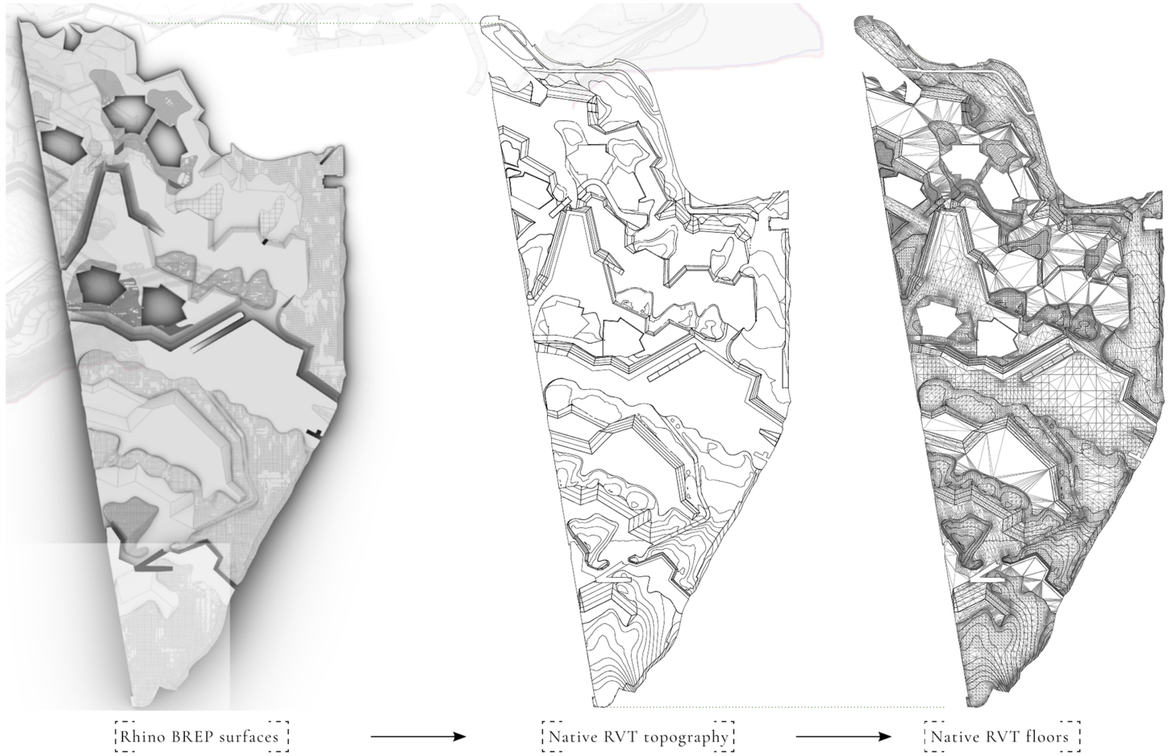
I was the BIM modeller and automation specialist, turning the urbanism team's CAD proposals directly into Revit elements on a very large project, and coordinating with the other projects in the cluster that made up the island.

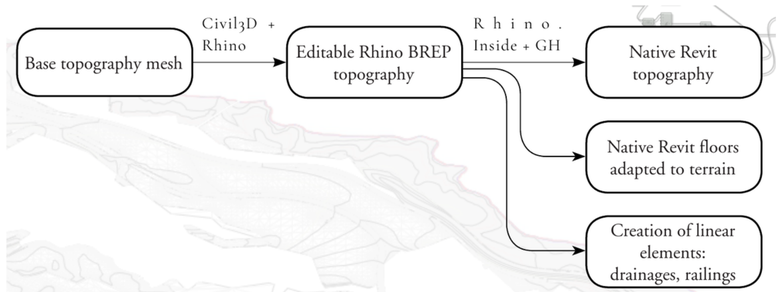
Civil 3D, Rhino, Rhino.Inside Revit, Grasshopper · 2023–2025 · Details under NDA



# From terrain mesh to native Revit elements

Civil 3D + Rhino → Rhino.Inside + Grasshopper → Revit




### THE BRIEF

The client needed a team with both BIM and complex-modelling skills to carry the earthworks and urbanism of a whole island inside Revit, working alongside the urban planners and architects on a live project.

### THE TERRAIN

The island's topography was built at the start, combining Civil 3D with Rhino meshing and surfaces. Every design decision after that was placed on top of this one shared terrain.

### INTO REVIT

Through Rhino.Inside and a set of Grasshopper scripts, the editable Rhino surfaces became native Revit topography, floors adapted to the terrain and linear elements like drainage and railings, with roads, paving and urban furniture on top.

PROJECT 03 | ACADEMIC | MACAD | IAAC

# Computational bucketing

Grouping a model's elements by any property, so anyone can read it in a browser

**PROBLEM**

Checking how a property spreads across a model meant clicking through elements one by one, and left out anyone without a Speckle account.

**APPROACH**

A Speckle Automate function groups elements by a chosen property and feeds a web viewer coloured by range.

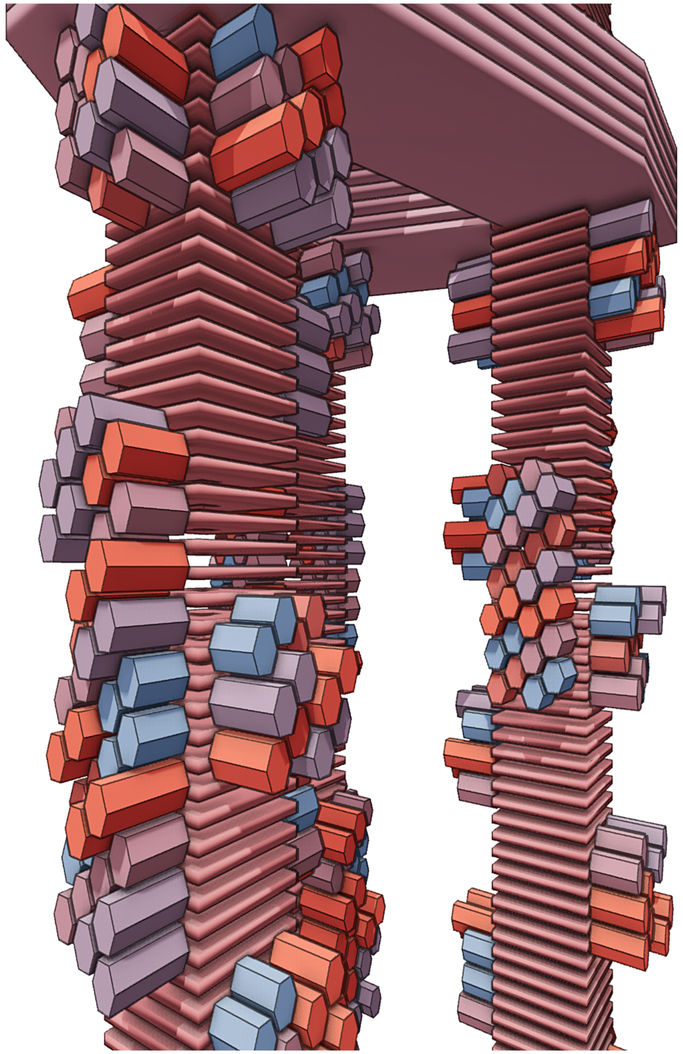
**RESULT**

A 25–30 minute model review now takes under three minutes.

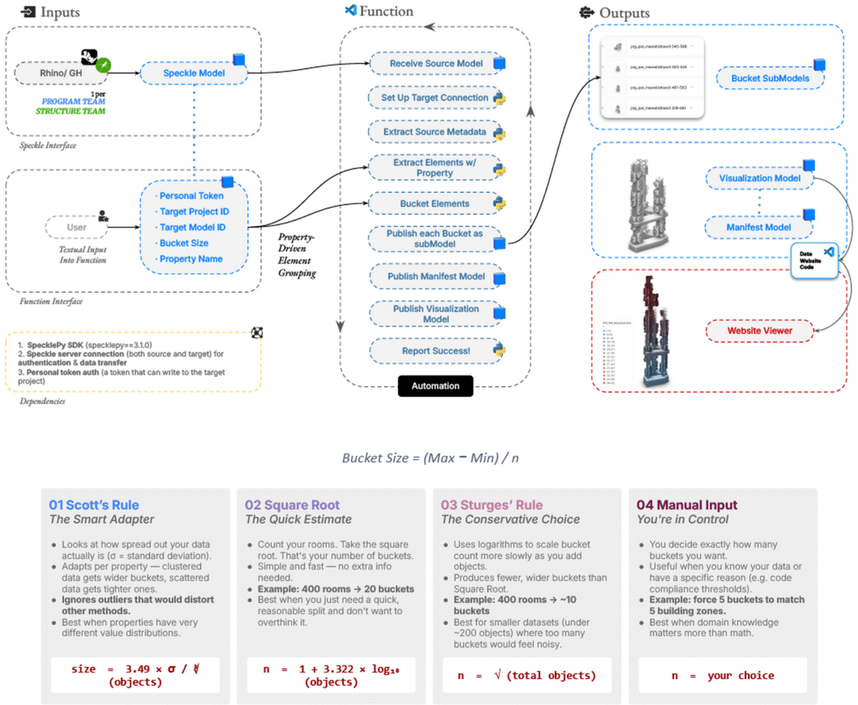
**COORDINATION**

A team of three, with the work shared roughly equally across the Automate function, the bucketing methods, the outputs published to Speckle, and the website that reads them.

Speckle Automate, Python, web viewer · 2026 · With Lakzhmy Mari Zaro and Emilie El Chidiac



# How it works




*Bucket Size = (Max − Min) / n*

**01 Scott's Rule**
*The Smart Adapter*

- Looks at how spread out your data actually is (σ = standard deviation).
- Adapts per property — clustered data gets wider buckets, scattered data gets tighter ones.
- **Ignores outliers that would distort other methods.**
- Best when properties have very different value distributions.

`size = 3.49 × σ / ∛(objects)`

**02 Square Root**
*The Quick Estimate*

- Count your rooms. Take the square root. That's your number of buckets.
- Simple and fast — no extra info needed.
- **Example: 400 rooms → 20 buckets**
- Best when you just need a quick, reasonable split and don't want to overthink it.

`n = 1 + 3.322 × log₁₀(objects)`

**03 Sturges' Rule**
*The Conservative Choice*

- Uses logarithms to scale bucket count more slowly as you add objects.
- Produces fewer, wider buckets than Square Root.
- **Example: 400 rooms → ~10 buckets**
- Best for smaller datasets (under ~200 objects) where too many buckets would feel noisy.

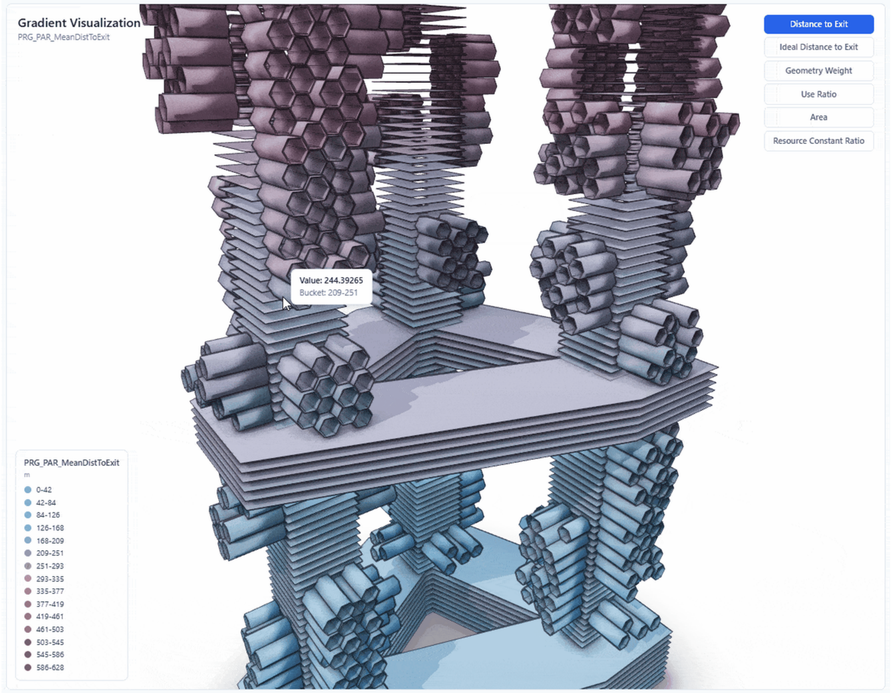
`n = √(total objects)`

**04 Manual Input**
*You're in Control*

- You decide exactly how many buckets you want.
- Useful when you know your data or have a specific reason (e.g. code compliance thresholds).
- **Example: force 5 buckets to match 5 building zones.**
- Best when domain knowledge matters more than math.

`n = your choice`

The pipeline, and the four ways to choose the number of buckets.




The website viewer: every element coloured by its value range.

### THE PIPELINE

On every new model version, the function flattens the object tree, reads a chosen numeric property from each element and groups the elements into range buckets, each published as its own sub-model.

### HOW MANY BUCKETS

Properties spread differently, so the count is a choice: Scott's rule adapts to the spread, square root is quick and general, Sturges' rule suits smaller datasets, and a manual number wins when domain knowledge matters more.

### FOR THE REVIEWER

A manifest and a gradient model feed a website whose viewer colours every element by its range. A 25–30 minute review now takes under three minutes, and nobody needs a Speckle account.

PROJECT 04 | ACADEMIC | MACAD | IAAC | OWN APS WORK

# Model data geometrization

Thermal, acoustic and air KPIs written into the model as elements, then checked in the cloud

### PROBLEM

Three studio teams produced geometry, but no one owned the data, so performance KPIs stayed in spreadsheets.

### APPROACH

We set naming and parameter standards, then used Rhino.Inside Revit to place KPI families and fill shared parameters automatically. I then built a dashboard that checks that standard on the AEC Data Model.

### RESULT

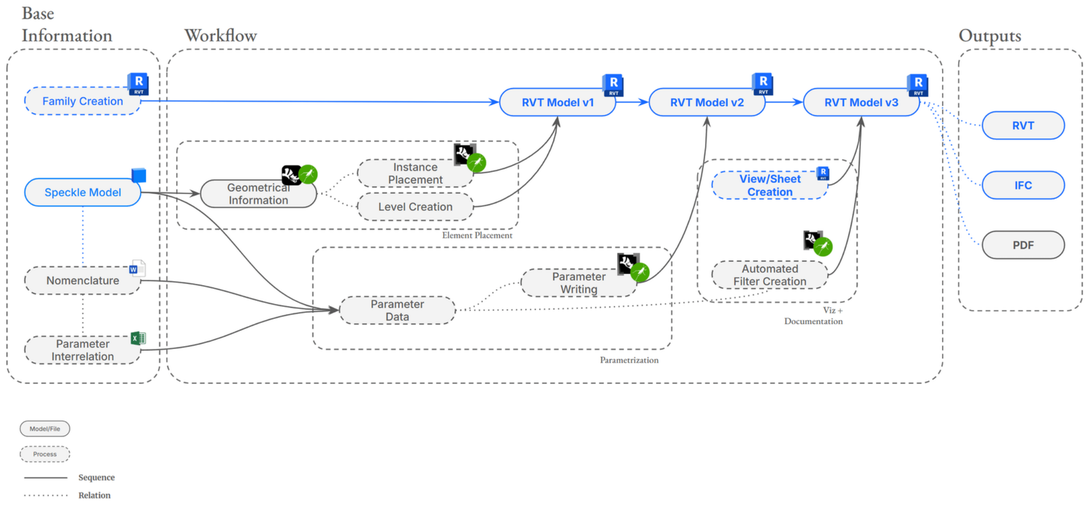
Thermal, acoustic and air-quality KPIs readable in the model, with filters, sheets and IFC export.

### COORDINATION

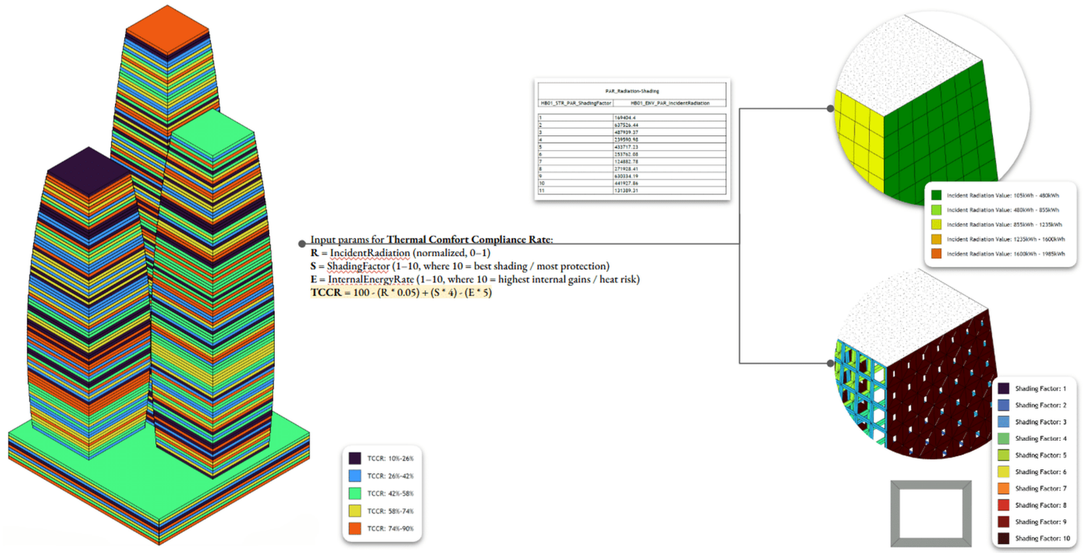
I worked at the planning level: the initial standards and the order of work across teams, while my teammates focused on execution and code. The dashboard extension is my own work.

Rhino.Inside Revit, Grasshopper, Speckle, Autodesk Platform Services, Python, Vue · 2026 · With Lakzhmy Mari Zaro and Emilie El Chidiac

# Making data spatial: three KPI families




Grasshopper reads the shared Speckle model; Rhino.Inside Revit places the families and fills their parameters.




Thermal compliance: each panel coloured by the KPI formula, read from radiation and shading.

### THE QUESTION

The data team draws no geometry, so how does its work show up in the model? We designed three parametric Revit families that turn performance indicators into elements you can see.

### THREE FAMILIES

The thermal panel maps solar radiation against shading. The noise orbit nests internal and external noise in one element. The pollutant sphere grows with the gap between polluted intake and clean air.

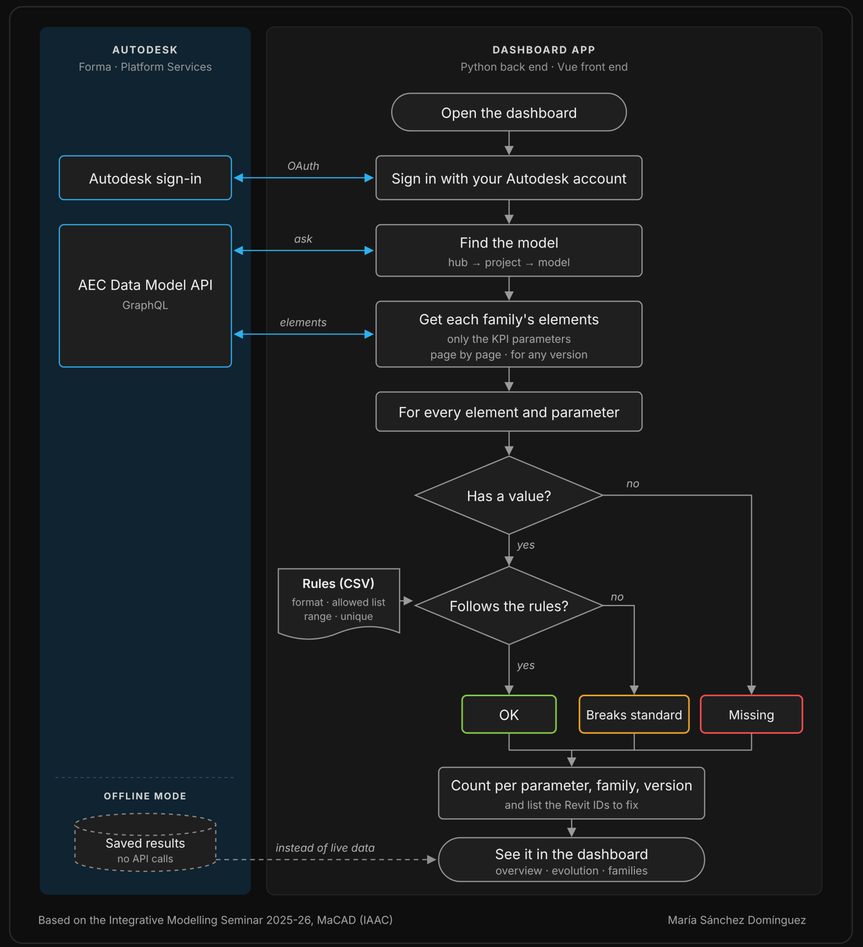
### THE WORKFLOW

Grasshopper reads positions from the shared Speckle model. Rhino.Inside Revit sets the levels, places the families and fills their shared parameters, then views, sheets and gradient filters follow, ready for RVT, IFC or PDF.

// APS dashboard · Python back end · Vue front end · AEC Data Model API

# Checking the data standard on the AEC Data Model

After the studio I rebuilt the data check as a web app on Autodesk Platform Services.
It reads the Revit model straight from Autodesk's cloud and measures, version by version, how much of each KPI family meets the standard.




Based on the Integrative Modelling Seminar 2025-26, MaCAD (IAAC)

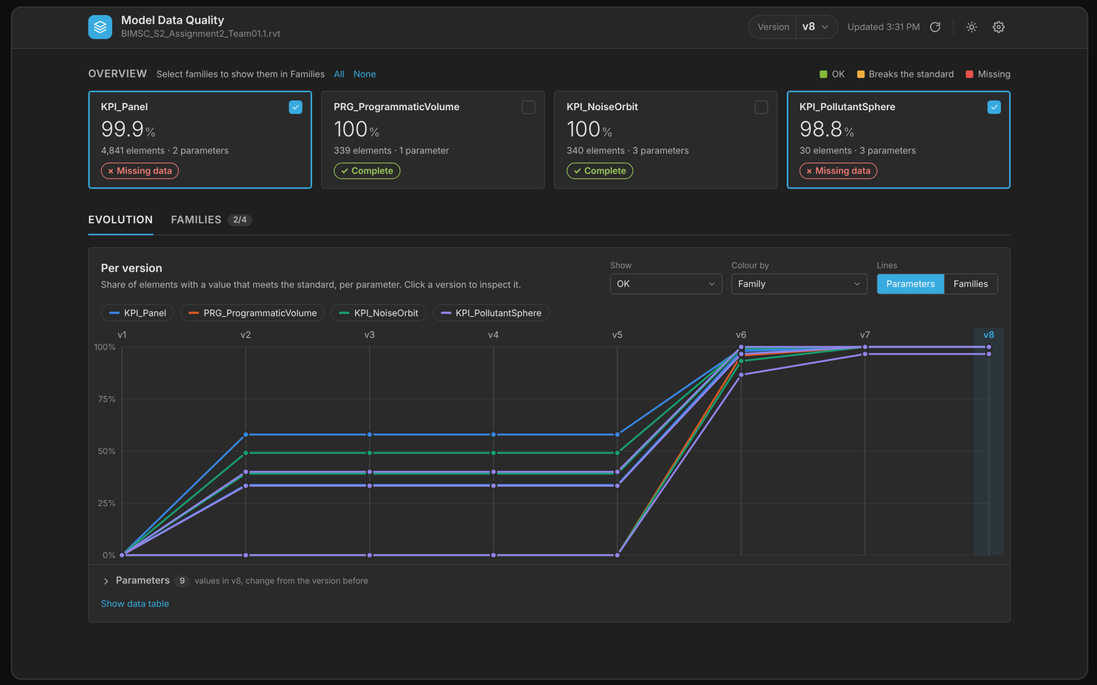
María Sánchez Domínguez

**01** **Sign in and find the model.** Autodesk sign-in through OAuth, then hub, project and model.

**02** **Ask for the data.** GraphQL queries to the AEC Data Model API fetch each KPI family's elements: only the KPI parameters, page by page, for any version.

**03** **Check every value.** Each parameter is tested against a rules file (CSV): format, allowed list, range, unique. It ends up OK, breaking the standard, or missing.

**04** **Show what to fix.** Results are counted per parameter, family and version, with the Revit IDs to fix. Saved results open offline, with no API calls.



PROJECT 05 | ACADEMIC | MACAD | IAAC

# Personal comfort copilot

Sensi edits a floor plan in plain language and shows the sensory trade-offs

**PROBLEM**

Floor plans get checked for structure, cost and energy, but not for how each room will feel to the person living in it.

**APPROACH**

It learns a user's sensory profile, scores every room with fixed rules, and lets them edit the plan in plain language. The LLM routes and explains; it never sets the score.

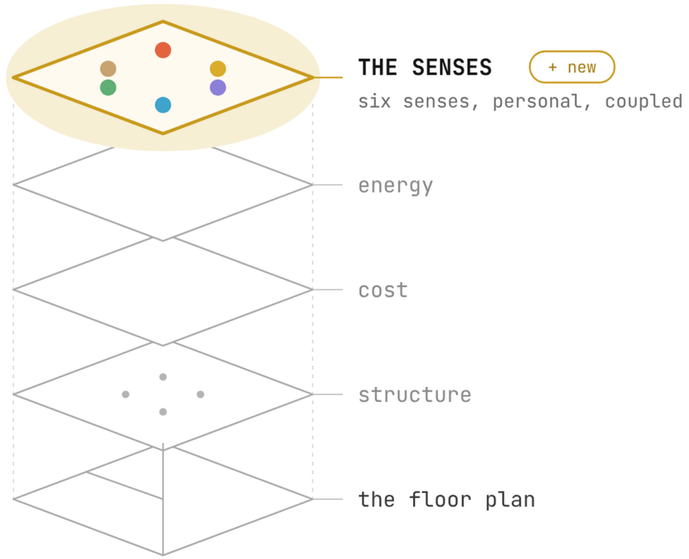
**RESULT**

Trade-offs like 'bigger window, more noise, colder room' become visible across a whole plan, re-scored live on every edit.

**COORDINATION**

Led by Emilie El Chidiac. I built the back end and front end and refined the LLM layer.

Python, LLM agents, web front end · 2026 · With Emilie El Chidiac, Lakzhmy Mari Zaro and Charles Abi Chahine



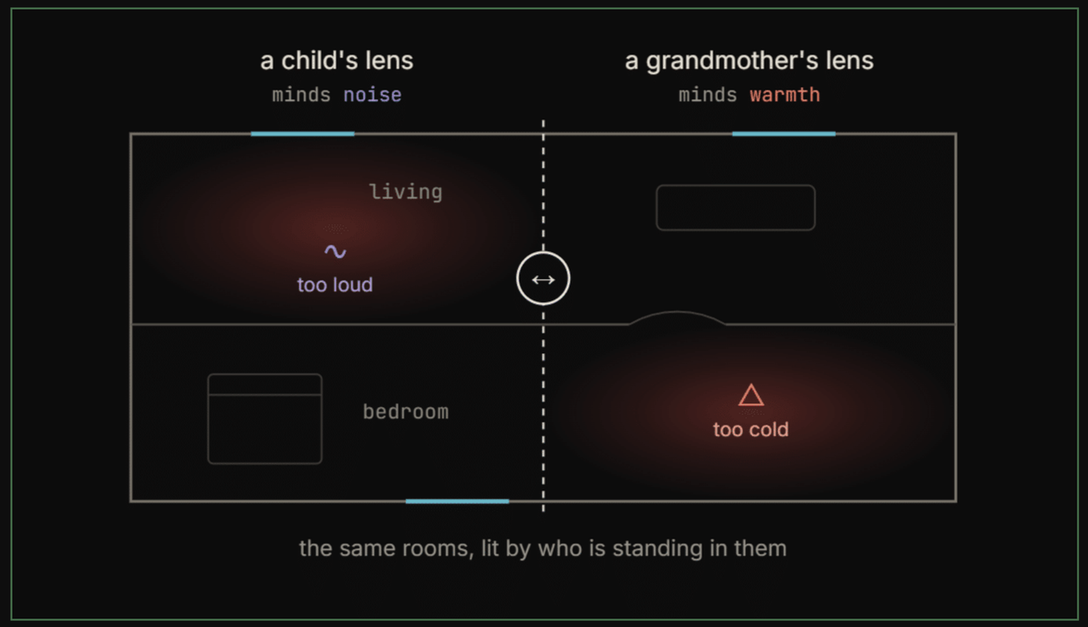

THE SENSORY LAYER IS

**personal**
scored for one person, never an average

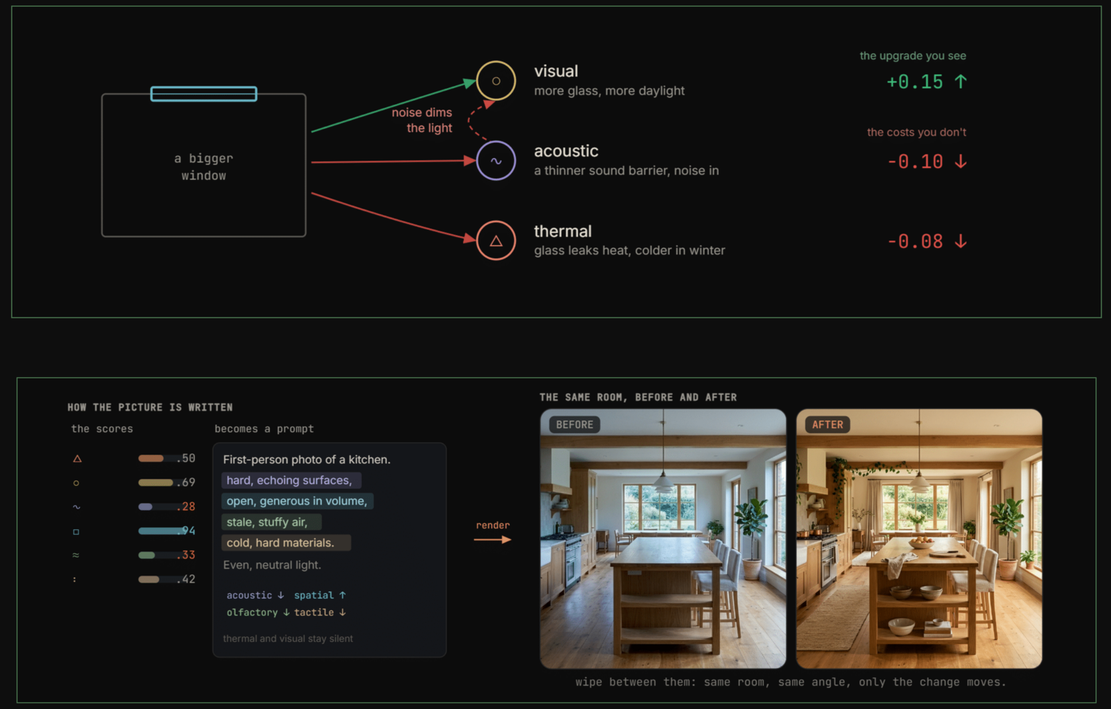
**honest**
no averaging; the worst sense carries the weight

**coupled**
the senses pull on each other

# Same plan, two people




One plan, seen through two personas.




The ripple of a bigger window, and the scores that write the render prompt.

### COMFORT IS PERSONAL

A child who minds noise and a grandmother who minds the cold flag different rooms in the same plan, so Sensi first builds a persona from a few questions and a moodboard.

### THE RIPPLE

The senses are coupled. A bigger window brings daylight but thins the sound barrier and leaks heat, and Sensi shows that whole chain, not just the upgrade. A room's score is half mean, half worst.

### FROM SCORE TO RENDER

Scoring follows fixed rules, so every edit can be traced. At the end the scores write a prompt and a vision model renders the room; only clearly good or bad senses add a phrase, so the render shows what changed.

# LET'S TALK

maria.sanchez.arch@gmail.com
mariasanchez.dev
linkedin.com/in/maría-sánchez-domínguez-arch